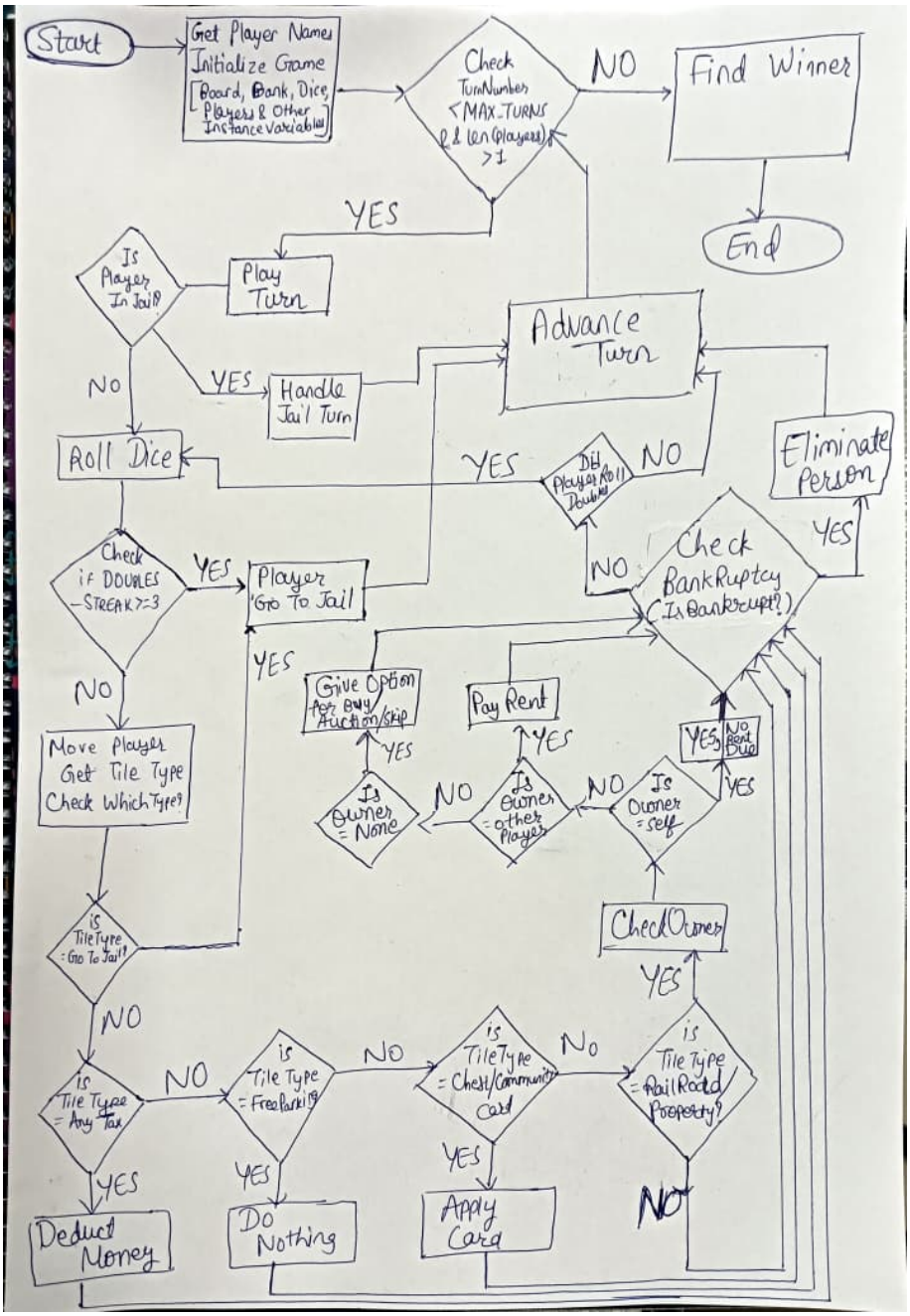
Design and Analysis of Software Systems
Assignment 2 Part 1 : White Box Testing

Meet Parekh (2024101122)

1.1 Control Flow Graph



The CFG above shows how the MoneyPoly game flows from start to finish. The game begins by taking player names as input and setting up the board, bank, dice, and all other parts of the game. It then enters the main loop, which keeps running as long as more than one player is still in the game and the number of turns has not hit the `MAX_TURNS` limit. If either condition fails, the game ends

by finding the player with the highest net worth and declaring them the winner. Inside each turn, we first check whether the current player is in jail. If they are, we handle the jail turn separately. If not, we roll the dice. We then check if the doubles streak has reached 3, which sends the player straight to jail. Otherwise, the player moves and we check what type of tile they landed on, handling it accordingly, whether that is a tax, a card draw, a property action, or nothing at all. After the tile is resolved, we check if the player has gone bankrupt. If they have, they are eliminated from the game. If not, we check whether they rolled doubles and therefore get to roll again. If they do not get another turn, we advance to the next player and repeat the loop.

1.2 Code Quality Analysis

Iteration 1: Removed unused imports from player, game, dice, and bank

I removed unused imports from `player.py`, `game.py`, `dice.py`, and `bank.py`. This cleaned up the files and removed the unused-import warnings reported by `pylint`.
Pylint score: 9.16/10

Iteration 2: Added missing module docstrings to all 10 modules

I added module-level docstrings to all source files in the project. This fixed the missing-docstring warnings and made the purpose of each file easier to understand.
Pylint score: 9.31/10

Iteration 3: Added docstrings for some functions and classes

I added missing docstrings to several functions and classes, mainly in `main.py`, `bank.py`, and `property.py`. This improved readability and removed the remaining documentation-related warnings.
Pylint score: 9.34/10

Iteration 4: Fixed code style issues

I fixed several formatting and style issues across the project. These changes included removing unnecessary parentheses, correcting boolean comparison style, fixing an unnecessary `else` after `return`, adding missing final newlines, and removing extra whitespace and tab spacing.
Pylint scores: 9.43/10, 9.46/10, 9.48/10

Iteration 5: Fixed code issues (unused variable, init fn, f-string, elif-after-break, except with error)

I fixed a few code-quality issues that affected clarity and correctness. I properly initialized the dice streak variable, removed an unnecessary f-string, corrected the menu branch after `break`, and replaced the bare exception handler in the input helper with specific exception types.
Pylint score: 9.54/10

Iteration 6: Fixed line length violations in cards.py by reformatting dictionaries

I reformatted the Chance and Community Chest card dictionaries in `cards.py` into a multi-line layout. This removed the line-length violations and made the card lists easier to read.
Pylint score: 9.91/10

Iteration 7: Fixed logical bugs (dice range, buy condition, find_winner, pay_rent transfer)

While fixing other issues and testing the game, I found some logical bugs that were not directly related to `pylint`. I corrected the dice range, changed the buy condition so exact balance is allowed, added rent transfer to the property owner, and fixed winner selection to use the highest net worth.
Pylint score: 9.91/10

Iteration 8: Refactored Property initialization, GameState and fixed too-many-arguments and too-many-instance-attributes warnings

This was the largest structural refactor in the project. I changed property creation to use a configuration dictionary instead of many constructor arguments, which reduced constructor complexity. I also grouped player jail-related data into a `jail_status` dictionary and game-level data into a `game_state` dictionary. In addition, I grouped card decks into a `decks` dictionary and split card handling into smaller helper methods such as `_handle_collect`, `_handle_pay`, `_handle_jail`, `_handle_move_to`, `_handle_birthday`, and `_handle_collect_from_all`. These changes reduced the number of direct attributes and constructor-style warnings reported by `pylint`.
Pylint score: 10.00/10

Iteration 9 / Error 0: Additional logical bugs found during code observation and playing

In the final cleanup pass, I fixed a few more logical issues found during code observation and gameplay testing. I made `Bank.collect()` ignore non-positive values, changed mortgage handling to use the correct payout method, credited the seller during trade, added an affordability check for paying the jail fine, and improved bankruptcy handling so the current player index stays valid after a player is removed. I also fixed the quote issue in the jail/status print statements.
Pylint score: 10.00/10

Result

After these iterations, the codebase reached a `pylint` score of 10.00/10.

1.3 White-Box Testing

Overview

White-box testing works by looking directly at the source code and writing tests that exercise every decision path. Every `if`, `elif`, and `else` branch. The test suite contains **119 collected items** across all modules. Most of the serious logical bugs in the codebase were already fixed in Iteration 7 and Iteration 9 (Task 1.2), so white-box testing found only a small number of remaining issues. The test file is at `whitebox/tests/test_whitebox.py` and is run with `pytest tests/test_whitebox.py -v` from the `whitebox/` directory.

Test Cases and Why Each One Is Needed

The table below lists every test case, the branch it targets, and a short explanation of why it is needed.

Test ID	Method / Branch	Why it is needed
bank.py		
TC_BANK_01	`collect()` positive	Verifies the normal path where money is added to the bank. Without this, we would never confirm that taxes and fines actually reach the bank.
TC_BANK_02	`collect()` zero/neg	The guard says negative amounts are silently ignored. This test confirms that passing 0 or a negative number leaves the bank balance unchanged.
TC_BANK_03	`pay_out()` success	Checks that the bank can pay out money and that its balance drops by the correct amount.
TC_BANK_04	`pay_out()` zero/neg	Ensures passing zero or a negative amount returns 0 and does not change the bank's balance.
TC_BANK_05	`pay_out()` insufficient	Confirms that a `ValueError` is raised when the bank is asked to pay more than it has.
TC_BANK_06	`pay_out()` boundary	Tests paying out the bank's exact full balance. This is an off-by-one boundary that could easily be missed.
TC_BANK_07	`give_loan()` deducts funds	**Bug found.** The bank was giving a player money without reducing its own reserves. This test catches that.
TC_BANK_08	`give_loan()` zero/neg guard	Confirms that requesting a loan of 0 or a negative amount does nothing to either the bank or the player.
player.py		
TC_PLAYER_01	`add_money()` normal	Checks that adding a positive amount increases the player's balance correctly.
TC_PLAYER_02	`add_money()` negative	Confirms that a negative amount raises a `ValueError` instead of silently reducing the balance.
TC_PLAYER_03	`deduct_money()` normal	Checks that deducting a positive amount decreases the player's balance correctly.
TC_PLAYER_03b	`deduct_money()` negative	Same guard as add, but for deduction. Confirms that a negative deduction is also rejected.
TC_PLAYER_04	add/deduct zero boundary	The guard is `amount < 0`, so zero must be accepted. This test confirms that passing 0 does not raise an error.

Test ID	Method / Branch	Why it is needed
TC_PLAYER_05	`is_bankrupt()`	Tests all three cases: positive balance is not bankrupt, zero is bankrupt, and negative is bankrupt. The boundary at zero is the critical case.
TC_PLAYER_06	`move()` lands on Go	When a player lands exactly on position 0, they collect $200. This test confirms that salary is awarded.
TC_PLAYER_07	`move()` no Go contact	Confirms that a normal move with no involvement of Go does not award any bonus money.
TC_PLAYER_08	`move()` wraps past Go	When a player wraps around the board but does not land on position 0, `move()` itself does not award salary. The game layer handles that separately.
TC_PLAYER_09	`go_to_jail()`	Confirms that all three jail state fields are set correctly: position, `in_jail`, and `jail_turns`.
TC_PLAYER_10	`net_worth()` properties	**Bug found.** `net_worth()` was only returning cash balance and ignoring the value of owned properties.
TC_PLAYER_11	game-layer Go salary	When a card moves a player backwards past Go, the game layer must award salary. This test checks that specific path.
property.py		
TC_PROP_01	`get_rent()` all branches	Checks all three rent outcomes in one test: mortgaged returns 0, partial group returns base rent, and full monopoly doubles the rent.
TC_PROP_02	`mortgage()`	Confirms that mortgaging works correctly and that trying to mortgage an already-mortgaged property returns 0 without changing anything.
TC_PROP_03	`unmortgage()`	Confirms the unmortgage cost is correct and that calling unmortgage on a property that is not mortgaged safely returns 0.
TC_PROP_04	`is_available()`	Tests all three availability states: free property returns True, owned returns False, mortgaged returns False.
TC_PROP_05	`all_owned_by()`	Checks that partial group ownership returns False and complete ownership returns True.
TC_PROP_06	`all_owned_by(None)`	Passing None as the player must return False without crashing.
TC_PROP_07	`get_owner_counts()`	Confirms that unowned properties are skipped and that per-player counts are correct for both single and split ownership.
dice.py		
TC_DICE_01	`roll()` streak tracking	Verifies that non-doubles resets the streak to 0 and that doubles increments it. Both branches of the if-doubles check must be exercised.
TC_DICE_02	`roll()` streak reaches 3	After three consecutive doubles the streak must equal 3, which is the trigger for sending a player to jail.
TC_DICE_03	`roll()` range 1–6	Confirms that dice values never go below 1 or above 6. This catches the bug from Iteration 7 where the range was 1–5.
board.py		

Test ID	Method / Branch	Why it is needed
TC_BOARD_01	`get_property_at()`	Checks that a valid position returns the correct property object and that a non-property position returns None.
TC_BOARD_02	`get_tile_type()`	Tests all 11 tile type strings. The game dispatches actions based on these strings, so any typo or missing case would silently break game logic.
TC_BOARD_03	`is_purchasable()`	Tests all four branches: unowned and clean returns True; owned, mortgaged, or non-property returns False.
TC_BOARD_04	`unowned_properties()`	Confirms 22 properties are unowned at the start and that the count drops correctly after one is purchased.
TC_BOARD_05	`is_special_tile()`	Confirms that known special positions return True and that a normal property or blank tile returns False.
TC_BOARD_06	`properties_owned_by()`	Confirms that the method returns only the properties belonging to the requested player and excludes properties owned by others.
cards.py		
TC_CARDS_01	`draw()` normal	Confirms that drawing from a non-empty deck returns a card and advances the index.
TC_CARDS_02	`draw()`/`peek()` empty	Both methods must return None on an empty deck instead of crashing.
TC_CARDS_03	`peek()` no advance	Peeking should show the next card without consuming it. The index must stay the same.
TC_CARDS_04	`draw()` cycling	After the last card is drawn, the next draw must wrap back to the first card via modulo.
TC_CARDS_05	`reshuffle()`	Confirms that reshuffling resets the index to 0.
TC_CARDS_06	`cards_remaining()`	Checks that the count is correct both before and after drawing cards.
game.py — Property Operations		
TC_GAME_01	`buy_property()`	Checks that buying succeeds when the player can afford it and fails when they cannot, leaving ownership and balance unchanged.
TC_GAME_02	`buy_property()` boundary	Balance exactly equal to price should succeed. This boundary was previously broken (it was using `<=` instead of `<`), fixed in Iteration 7.
TC_GAME_03	`buy_property()` ownership	Documents that the ownership re-check lives in the UI layer, not inside `buy_property()` itself.
TC_GAME_04	`pay_rent()` branches	Checks all three rent paths: unowned means no payment, mortgaged means no payment, and normally owned means money transfers from payer to owner.
TC_GAME_05	`mortgage_property()`	Checks that the wrong owner is rejected, that mortgaging by the correct owner succeeds, and that trying to mortgage again fails.
TC_GAME_06	`unmortgage_property()`	**Bug found.** The mortgage was being lifted before the balance check. This test catches all four branches: wrong owner, insufficient funds, not mortgaged, and success.
game.py — Jail Logic		

Test ID	Method / Branch	Why it is needed
TC_GAME_07	jail free card	Confirms that using a Get Out of Jail Free card releases the player and removes the card from their hand.
TC_GAME_08	voluntary fine	Checks that paying the fine releases the player and deducts the money, and that a player who cannot afford it stays in jail.
TC_GAME_09	mandatory release	After three jail turns the player must be released. The balance assertion only checks that the fine was deducted; later tile effects are not part of this test.
game.py — Turn & Tile Logic		
TC_GAME_10	`advance_turn()`	Checks that the player index wraps around correctly and that the turn number always increments.
TC_GAME_11	doubles keeps turn	When a player rolls doubles the turn index must not advance. The same player should roll again.
TC_GAME_12	three doubles jail	Three consecutive doubles must send the player to jail immediately without moving them.
TC_GAME_13	special tiles	Tests go-to-jail, income tax, luxury tax, and free parking in a single test, confirming correct money and state changes.
TC_GAME_14	chance tile	The chance branch in `_resolve_tile()` must draw and apply a card from the chance deck.
TC_GAME_15	community chest tile	The community chest branch must draw and apply a card from the community chest deck.
TC_GAME_16	railroad tile	The railroad branch routes to the property tile handler. This test confirms no crash and correct balance when no property object exists at the tile.
TC_GAME_17	property tile skip/own/other	Tests all three ownership states: unowned and skipped, owned by self (no rent), and owned by another player (rent paid).
TC_GAME_18	property tile buy path	Choosing to buy when landing on an unowned property should transfer ownership and deduct cost.
TC_GAME_19	property tile auction path	Choosing to auction should run the bidding process and transfer ownership to the highest bidder.
game.py — Card Actions		
TC_GAME_20	collect/pay/None/unknown	Tests that collect adds money, pay removes it, None does nothing, and an unrecognised action is silently ignored.
TC_GAME_21	move_to/jail/jail_free	Tests all movement card actions: moving to a lower position awards Go salary, higher position does not, jail sends the player to jail, and jail-free gives a card.
TC_GAME_22	birthday action	All other players pay the birthday amount. A player who cannot afford it is skipped without going negative.
TC_GAME_23	collect_from_all action	Same logic as birthday but a different dispatch key. Both branches must be exercised separately to confirm the dispatcher is correct.
game.py — Auction & Trade		

Test ID	Method / Branch	Why it is needed
TC_GAME_24	`auction_property()`	Checks four auction paths: a valid bid wins, a bid below the minimum increment is rejected, a bid above the player's balance is rejected, and passing results in no winner.
TC_GAME_25	auction minimum bid boundary	A bid exactly at the minimum increment should be accepted. This is a boundary case that off-by-one errors could break.
TC_GAME_26	`trade()` all paths	Checks that the wrong seller fails, an unaffordable trade fails, a normal trade succeeds with correct money and ownership transfer, and a zero-cash gift also succeeds.
TC_GAME_27	trade mortgaged property	Trading a mortgaged property should succeed and the mortgage state should be preserved on the new owner.
game.py — Bankruptcy & Winner		
TC_GAME_28	`_check_bankruptcy()`	Confirms that a bankrupt player is removed from the game and that all their properties are released back to the board.
TC_GAME_29	bankruptcy index (first)	When the first player (index 0) is removed, the current index must not become -1. Python negative indexing would silently cause the wrong player to take the next turn.
TC_GAME_30	bankruptcy index (middle)	When a middle player is removed the index must stay within the bounds of the remaining list.
TC_GAME_31	bankruptcy index (last)	When the player at the last index is removed the index must be adjusted so it does not go out of bounds.
TC_GAME_32	`find_winner()` empty	An empty player list must return None without crashing.
TC_GAME_33	`find_winner()` richest	Must return the player with the highest net worth. This was broken in Iteration 7 where `min()` was used instead of `max()`.
TC_GAME_34	all players eliminated	When every player goes bankrupt the game must end cleanly and find_winner must return None.
ui.py		
TC_UI_01	`safe_int_input()` valid	The try branch: a valid integer string is converted and returned.
TC_UI_02	`safe_int_input()` ValueError	The except ValueError branch: a non-integer string should return the default value.
TC_UI_03	`safe_int_input()` EOFError	The except EOFError branch: an end-of-file signal (common in automated testing) should also return the default value.
main.py		
TC_MAIN_01	KeyboardInterrupt	Confirms that pressing Ctrl-C during a game is caught and the program exits with a message instead of crashing.
TC_MAIN_02	ValueError	Confirms that a bad game setup (e.g. empty player list) is caught and printed as an error message instead of crashing.

Test ID	Method / Branch	Why it is needed
game.py — run() Loop		
TC_RUN_01	one player break	When only one player remains the while loop must break immediately without calling `play_turn()`.
TC_RUN_02	winner branch	After the loop ends with at least one player alive, the if-winner branch must fire and declare that player.
TC_RUN_03	no winner branch	When all players are eliminated the else branch must fire and the game ends with no winner.
TC_RUN_04	MAX_TURNS limit	The loop must exit when the turn counter reaches `MAX_TURNS`, even if multiple players are still in the game.
game.py — interactive_menu() and _menu_*		
TC_MENU_01	choice 0 exits	Choosing 0 must break the while True loop and return immediately.
TC_MENU_02	choice 1 standings	Choosing 1 must call `print_standings()` with the current player list.
TC_MENU_03	choice 2 board ownership	Choosing 2 must call `print_board_ownership()` with the board.
TC_MENU_04	choice 6 loan issued	Choosing 6 with a positive amount must call `give_loan()` and credit the player.
TC_MENU_05	choice 6 zero loan skipped	If the amount entered is 0, the if-amount->0 guard must prevent any loan from being issued.
TC_MENU_06	mortgage no properties	If the player owns nothing the early-exit branch must fire and nothing should happen.
TC_MENU_07	mortgage valid index	Selecting a valid index must call `mortgage_property()` on the correct property.
TC_MENU_08	mortgage invalid index	An out-of-bounds index must be silently ignored.
TC_MENU_09	unmortgage no mortgaged	If none of the player's properties are mortgaged the early-exit branch must fire.
TC_MENU_10	unmortgage valid index	Selecting a valid index must call `unmortgage_property()` on the correct property.
TC_MENU_11	unmortgage invalid index	An out-of-bounds index must be silently ignored.
TC_MENU_12	trade no other players	In a single-player game the early-exit branch must fire immediately.
TC_MENU_13	trade invalid partner index	An out-of-bounds partner selection must return early without starting a trade.
TC_MENU_14	trade no properties	If the player owns nothing the no-properties guard must fire and no trade starts.
TC_MENU_15	trade invalid property index	An out-of-bounds property selection must return early without starting a trade.
TC_MENU_16	trade full happy path	A complete valid trade through the menu must transfer ownership and money correctly.

Errors Found and Fixed

Well, Iterations 7 and 9 from Task 1.2 already resolved the bulk of the logical issues in the codebase before white-box testing began. The test suite confirmed those fixes and found four additional bugs, each fixed in a separate commit.

Error 0 (Iteration 7) : Four bugs found during gameplay and code review. The dice range was 1–5 instead of 1–6 in `dice.py`. In `game.py`, `buy_property()` was using `<=` so a player with exactly enough money was wrongly blocked; `pay_rent()` deducted from the payer but never added to the owner; and `find_winner()` used `min()` instead of `max()`, declaring the poorest player as winner.

Error 0 (Iteration 9) : More bugs found during observation. In `bank.py`, `collect()` was not ignoring negative amounts. In `game.py`: `mortgage_property()` was calling `bank.collect(-payout)` instead of `bank.pay_out(payout)`; `trade()` deducted from the buyer but never credited the seller; `_handle_jail_turn()` freed the player without checking if they could afford the fine; and `_check_bankruptcy()` could set the current index to -1 when the first player was removed. Quote-style syntax errors in f-strings in `game.py` and `ui.py` were also fixed.

Error 1 : `Bank.give_loan()` in `bank.py` was adding money to the player but never subtracting it from the bank's own reserves (`self._funds`). The fix was to add `self._funds -= amount` so the bank's balance actually reflects the loan.

Error 2 : `Player.net_worth()` in `player.py` returned only `self.balance`, completely ignoring owned properties. The fix was to sum the mortgage value of all properties the player owns and add that to the cash balance.

Error 3 : `Game.unmortgage_property()` in `game.py` called `prop.unmortgage()` before checking whether the player could afford the cost. Because `prop.unmortgage()` immediately lifts the mortgage flag as a side effect, a failed affordability check still left the property in an unmortgaged state. The fix was to check `is_mortgaged` and the player's balance first using `mortgage_value()`, and only call `prop.unmortgage()` after all checks pass.

Error 4 : The test for mandatory jail release was asserting that the player's balance equals exactly $150 after paying a $50 fine from $200. This was wrong because after being released the player rolls the dice and lands on a tile, which can add or subtract money unpredictably. The fix was to remove the balance assertion entirely and only assert that `in_jail` is `False`, which is the one thing this test is actually responsible for checking.